The Essential of Chapter 4

Query processing cut down to what the two mock exams ask: pipeline breakers, the passes of an external sort, the cost of each join, `Batches`, and which join wins

Advanced Data Management

2026-10-29

How to read this document

This is chapter 4 with everything removed that neither mock exam touches. Six sections, each ending in a *Mock drill* box that says which mock question lives there and the shape of the answer the rubric wants. Read it in order: every term is defined before it is used. Notation: B is the number of pages of a file, B_R and B_S the pages of the two join inputs, $|R|$ the number of *rows* of R, M the number of buffer pages an operator may use (`work_mem` in PostgreSQL, 8 KB per page), h the height of a B$^+$-tree. Database facts the mocks reuse: `customers` 10,000 rows on 73 pages, `orders` 500,000 rows on 3,185 pages; default `work_mem` 4 MB = 512 pages.

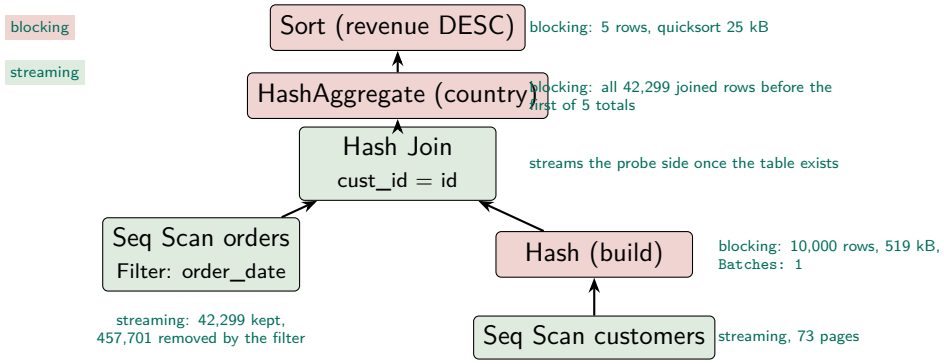
Left out on purpose: the merge-sort refresher (§1), the table of simple operators (§3), the tiny customers/orders traces of each join (§5.1–5.4 examples), the merge-join and nested-loop plan listings (§7.2), the Lab 4 companion and its debugging box (§8), and classic problems P2, P6 and P8.

1 The Executor: Iterators and Pipeline Breakers

Concept: Iterator model, pipelining, pipeline breaker

A physical plan is a tree of operators. Every operator implements `open()`, `next()` and `close()`: the parent calls `next()` on its children to pull one tuple at a time, so requests flow down the tree and tuples flow up. An operator is **pipelined** (streaming) when it can pass a tuple on as soon as it has seen it: a sequential scan, a filter, a projection, the probe side of a join. A **pipeline breaker** (blocking operator) must consume *all* of its input before it can emit *any* output. The three that matter: **Sort** (the last tuple might be the smallest), the **Hash build** of a hash join (the whole table must exist before the first probe), and **hash-based aggregation** (a `SUM` per group is not final until the last row has arrived).

A breaker **materialises** its input, in memory when it fits in `work_mem` and otherwise in temporary files on disk; that is where the I/O of a plan concentrates, so reading a plan starts with finding the breakers. In `EXPLAIN ANALYZE`, `actual time=a..b` gives the first-row time a and the last-row time b: a breaker shows $a \approx b$, a streaming operator shows a wide gap.



The tree is mock v2's December report: `orders` joined to `customers` on `cust_id = id`, filtered on `order_date`, grouped by `country`, sorted by revenue. Read it bottom-up: the customers scan streams into **Hash**, which must finish before the join can produce a row; the orders scan streams, filtering as it reads; the join probes and passes each order up at once; the aggregate and the sort each wait for everything below them. Two of the five nodes stream, so the first output row appears only at the very end, as it must for a report with `GROUP BY` and `ORDER BY`.

Mock drill

Mock v2 C1: draw the five nodes (Sort, HashAggregate, Hash Join, Seq Scan orders, Hash over Seq Scan customers), label each blocking or streaming with one reason each. Quiz A1–A2, B4: name three breakers; the one aggregate that is *not* a breaker is GroupAggregate over sorted input (§6). Full chapter: §2.2 page 3, §7.1 page 12.

2 External Merge Sort

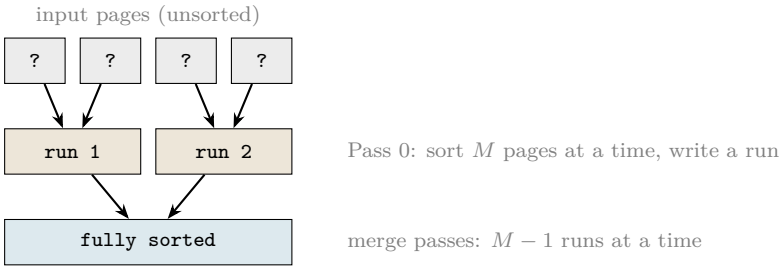
Mechanism: External merge sort with M buffer pages

A file of B pages with $B > M$ cannot be sorted in memory, so it is sorted in pieces.

- **Pass 0, run generation.** Read M pages, sort them in memory, write them back as one **sorted run** of M pages. Repeat: $\lceil B/M \rceil$ runs.
- **Merge passes.** Keep one buffer page for the output being assembled and use the other $M-1$ as inputs, one per run: a $(M-1)$-way merge turns $M-1$ runs into one. Each merge pass divides the run count by $M-1$; stop when one run remains.

Every pass reads and writes the whole file: $2B$ page I/Os per pass. The fan-in is $M-1$, not M, because a page is the unit of writing, so one buffer

must hold the merged output.



Concept: The sort cost formula

$$\text{runs} = \lceil B/M \rceil, \qquad \text{fan-in} = M - 1,$$
$$\text{merge passes} = \lceil \log_{M-1} \text{runs} \rceil, \qquad \text{total I/O} = 2B\,(1 + \text{merge passes}).$$

The "1" is pass 0. Never compute the logarithm: write the run count, divide by $M-1$ (rounding up) until you reach 1, and count the divisions; k merge passes are needed exactly when $(M-1)^{k-1} < \text{runs} \le (M-1)^k$. Two thresholds follow:

- **Two passes** (one merge pass) when all runs merge at once: $\lceil B/M \rceil \le M - 1$, which is $M \approx \sqrt{B}$ plus one; check the boundary by hand.
- **One pass** when the file fits: $M \ge B$. The sort reads B pages once and writes nothing.

Key insight. Passes are a ceiling of a logarithm, so memory buys whole passes, each worth $2B$. One extra buffer page at the right boundary removes a pass; a hundred extra pages in the middle of a regime remove nothing.

Worked example: The four sorts the mocks ask for

Mock 1, 1.1: $B = 800$**,** $M = 21$**.** Runs $\lceil 800/21 \rceil = 39$. Fan-in 20: $39 \to 2 \to 1$, two merge passes. **3 passes**, I/O $2 \times 800 \times 3 = 4{,}800$. Trap: 39 is the run count; 2 passes would need $39 \le 20$.

Mock v2 A5: $B = 6{,}000$**,** $M = 100$**.** Runs 60, fan-in $99 \ge 60$: one merge pass. **2 passes**, 24,000 I/Os. Trap: 60 is the run count; 1 pass would need $M \ge 6{,}000$.

Mock v2 B1: $B = 1{,}200$**,** $M = 25$ **then 50.** $M = 25$: 48 runs, fan-in 24, $48 \to 2 \to 1$, 3 passes, 7,200 I/Os (3,600 reads and 3,600 writes). $M = 50$: 24 runs, fan-in $49 \ge 24$, 2 passes, 4,800. Doubling memory cut the I/O by a third, not a half: it removed exactly one pass, and no further doubling removes another, since one merge pass is the floor until $M \ge B = 1{,}200$ pages = 9.6 MB, where the sort becomes a single in-memory pass. The one-merge-pass threshold was already $M = 36$ (34 runs against a fan-in of 35; $M = 35$ gives 35 runs against 34).

Mock 1, 3.4(a): $B = 1{,}000{,}000$**,** $M = 8{,}000$**.** 125 runs, fan-in $7{,}999 \ge 125$: 2 passes, 4,000,000 I/Os, about 32 GB of traffic, over a minute at 500 MB/s. Since $\sqrt{10^6} = 1{,}000$, any M above about 1,000 is in the two-pass regime.

In PostgreSQL `work_mem` is M. A sort that fits prints `Sort Method: quicksort Memory: 205kB`; one that spills prints `Sort Method: external merge Disk: 8184kB`. `work_mem` is allocated *per sort or hash operation, per connection*: a 256 MB setting with fifty queries each doing two sorts is 25 GB, which is why it is set modestly and raised with `SET LOCAL` for a known heavy statement.

Mock drill

Mock 1, 1.1 and **Mock v2 A5**: letter plus one line "runs = $\lceil B/M \rceil$, fan-in $M-1$, divide until 1, add pass 0"; name the trap (the run count is not the pass count). **Mock v2 B1**: data, the three formulas, the two computations with units, then the decision: "a third, not a half; passes are a logarithm; single pass needs $M \geq B$, 9.6 MB". **Mock 1, 3.4(a)**: runs, passes, I/O, and the $\sqrt{B}$ remark. Full chapter: §4.2–4.3 pages 5–6, classic P1–P2.

3 Nested-Loop Joins

Concept: Outer, inner, equi-join

A join combines rows of R and S where a condition holds; when the condition is $R.x = S.y$ it is an **equi-join**. In a nested loop the **outer** input is read once and drives the loop; the **inner** is re-read at each step of the outer. All three variants test the predicate directly, so they are the only joins that accept *any* condition (`<`, `BETWEEN`, `LIKE`).

Mechanism: Three variants, three costs

Simple nested loop. For each *row* of R, scan all of S: $B_R + |R| \cdot B_S$. The inner is read once per row of the outer, which is absurd for any real table.

Block nested loop. Read R in chunks of $M-2$ pages (one buffer holds the current S page, one the output); for each chunk scan S once and join every row of the chunk with every row of S:

$$B_R + \left\lceil \frac{B_R}{M-2} \right\rceil \cdot B_S.$$

The inner is read once per *chunk* of the outer. Put the **smaller input outside**: fewer chunks, fewer rescans. When the whole outer fits in one chunk ($M - 2 \geq B_R$) the cost is $B_R + B_S$, each input read once.

Index nested loop. If S has a B^+-tree on the join key, for each row of R descend the index instead of scanning S: $B_R + |R| \cdot (h + \text{matches})$, one descent of h pages plus one heap page per matching row. In a plan it is `Nested Loop` over an inner `Index Scan` with `loops=`$|R|$: the inner ran once per outer row. Unbeatable when $|R|$ is tiny (`WHERE c.id = 42`: one descent, about $1 + (3 + 1)$ pages, where a hash join must read all 3,185 pages of `orders`); a disaster when $|R|$ is large (the class example's

`loops=20000` takes 641 ms against 232 ms for the hash join; the same plan with `c.id < 10` runs in 0.26 ms).

Worked example: Block nested loop, the two mock instances

Mock 1, 1.5: $B_R = 300$, $B_S = 3{,}000$, $M = 32$. Chunks $\lceil 300/30 \rceil = 10$; cost $300 + 10 \times 3{,}000 =$ **30,300**. The traps are the other three letters: 9,300 uses chunks of 100 pages (an M of 102); 900,300 is the simple nested loop with 300 *rows*, $300 + 300 \times 3{,}000$; 3,300 is $B_R + B_S$, the cost when the outer fits in one chunk, which needs $M \geq 302$, or a hash join.
Mock v2 B3(a), (c): $B_R = 80$, $B_S = 3{,}200$, $M = 10$ **then 100.** $M = 10$: chunks $\lceil 80/8 \rceil = 10$, cost $80 + 10 \times 3{,}200 =$ **32,080**; the inner is read ten times. $M = 100$: $\lceil 80/98 \rceil = 1$ chunk, $80 + 3{,}200 =$ **3,280**: the whole outer sits in memory and S is read once.

Mock drill

Mock 1, 1.5: letter, the chunk computation ($M - 2$, rounded up), the sum, and the three traps named. **Mock v2 B3(a), (c)**: the same with $M = 10$ and $M = 100$; say in words why $M = 100$ reads S once. **Mock v2 D2, nested-loop half**: the `c.id = 42` lookup, $1 + (h + 1)$ pages against 3,185, plus "the only algorithm for non-equi joins". Full chapter: §5.2 pages 7–8 and the "$|R|$ versus B_R" warning in §9.

4 Sort-Merge Join and Hash Join

4.1 Sort-merge join

Mechanism: Sort-merge join

Sort R and S on the join key (external merge sort, §2), then **merge**: walk both sorted inputs with one pointer each, advance the smaller key, and when the keys are equal emit every pairing of the equal-key rows. Cost $= \text{sort}(R) + \text{sort}(S) + (B_R + B_S)$, where each sort is $2B$ (1 + merge passes) and the final term is the one linear merge pass. The output comes out sorted on the join key, and the merge needs no hash table and no memory beyond its two input pages.

It wins when the sorts are free: both inputs already arrive sorted (index scans on `customers(id)` and `orders(cust_id)` deliver key order), or the query wants `ORDER BY` or `GROUP BY` on the join key anyway. It also handles range joins (`R.x < S.y`), which hashing cannot. It loses when neither input is sorted and memory is short: two multi-pass external sorts before the first output row. In the class example the merge-join plan sorts 500,000 orders (`external merge Disk: 8184kB`), its first row appears at 247 ms, and the join takes 432 ms against 232 ms for the hash join that had no sort to wait for.

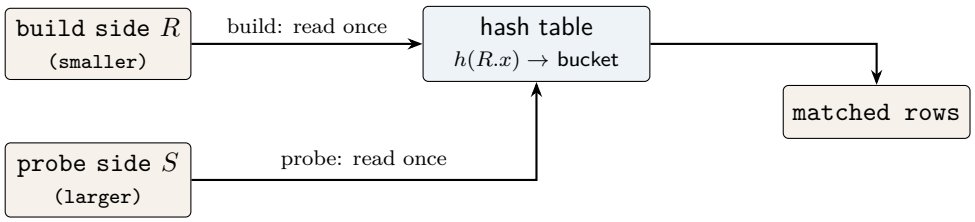
4.2 Hash join

Mechanism: Hash join: build, probe, Grace

1. **Build.** Read the *smaller* input R once and insert every row into an in-memory hash table keyed on $h(R.x)$. This is the pipeline breaker.
2. **Probe.** Read the larger input S once; for each row look in bucket $h(s.y)$ and emit the matches. Streaming; the output is in S's file order, with no ordering guarantee.

If the build side fits in `work_mem`: cost $B_R + B_S$, each input read exactly once, the minimum any join can cost. If it does not: **Grace hash join**. Read R once, hash each row with a partitioning function h_1 into k partitions written to disk; do the same to S with the same h_1. A row of R can only match rows of S in the partition with the same h_1 value, so join partition i of R with partition i of S, each pair small enough to build in memory. Cost: partition both (read and write, $2(B_R + B_S)$) then join (read both once, $B_R + B_S$): **about** $3(B_R + B_S)$.

In a plan, `Hash ... Buckets: 16384 Batches: 1 Memory Usage: 519kB`: `Batches` is the number of Grace partitions. **1 means the build side fit in `work_mem`; 8 means both inputs were split into 8 partitions**, the first kept in memory and the other 7 written to temporary files, then joined pair by pair. The spill shows as `temp written=` on the Hash node (build partitions) and `temp read=... written=...` at the top (probe rows of batches 2–8, written once and read back once). These counters are not the buffer pool; they are a breaker spilling.



Worked example: The hash-join numbers of both mocks

Mock v2 A4: which side builds? `customers`, 73 pages (519 KB in the real plan): it fits in one batch of the default 4 MB. `orders`, 3,185 pages = 25 MB, would not, and would be split into batches on disk. Build on the smaller side so that each table is read once. "Either, same cost" is true only when both fit; "both hashed" describes Grace, which is what we avoid.

Mock v2 B3(b): $B_R = 80$, $B_S = 3{,}200$, $M = 10$. $80 > 10$: the build does not fit, so Grace: $3 \times 3{,}280 =$ **9,840** (feasible: 9 partitions of about 9 pages each fit in 10 buffers). Against the block nested loop's 32,080 that is 3.3× cheaper. **B3(c),** $M = 100$: $80 \leq 100$ fits, in-memory variant $B_R + B_S = 3{,}280$; the block nested loop also reads S once at this memory, so **the two tie**: with enough memory both read each input exactly once.

Mock 1, 3.4(b): `users` 30,000 pages, `events` 1,000,000 pages, no

index, `work_mem` 8,000 pages. Hash join, build on `users`. 30,000 > 8,000: Grace, about $\lceil 30{,}000/8{,}000 \rceil = 4$ batches (PostgreSQL rounds to a power of two), I/O $3 \times 1{,}030{,}000 =$ **3,090,000**, plan line `Hash ... Batches: 4`. Compare: sort-merge = 4,000,000 (sort events, §2) +120,000 (sort users, 2 passes) +1,030,000 = 5,150,000; block nested loop with users outside $= 30{,}000 + 4 \times 10^6 = 4{,}030{,}000$. Hash wins. A filter shrinking `users` to 3,000 pages would make the build fit: $B_R + B_S \approx 1{,}030{,}000$.

Mock v2 C5: run 3 with `work_mem = '64kB'`. The customers table needed 519 kB; at 64 kB it no longer fits, so `Buckets: 4096 Batches: 8 Memory Usage: 80kB`: a Grace hash join with 8 partitions. `temp written=28` on the Hash node is the 7 spilled build partitions, about 4 pages each; `temp read=183 written=183` at the top is the `orders` rows of batches 2–8. Same plan shape, memory moved to disk, 62.3 to 71.5 ms (+15%; tiny temp files, served by the OS cache). Recommendation: `work_mem` above the build's need with margin, at least 1 MB here, so the default 4 MB is right; set it per statement with `SET LOCAL`, never globally, because it multiplies across every concurrent sort and hash.

Mock drill

Mock v2 A4: letter (customers), one line: "the build side is read into memory; $B_R + B_S$ holds only while it fits; 73 pages fit, 3,185 would batch". **Mock v2 B3(b), (c)**: does R fit in M, which variant, $3(B_R + B_S)$ or $B_R + B_S$, the ratio against the nested loop, the tie at $M = 100$. **Mock 1, 3.4(b)**: which join, build side, fits in `work_mem`?, batches, I/O, the two alternatives, the plan line. **Mock v2 C5**: what changed in the Hash node, `Batches: 8`, the two temp lines, `SET LOCAL`. Full chapter: §5.3–5.4 pages 8–10.

5 Which Join Wins, and When Each Is a Disaster

No algorithm dominates: the winner depends on the join condition (equality or not), the sizes (is one side tiny?), the indexes (is there one on the inner key?), the order already present or required, and the memory available for a build or a sort. Each algorithm is the best choice in one of the other two's disasters, which is why a database keeps all three. The optimiser (chapter 5) costs them per join with the estimated sizes: the class example runs one 500,000-row join three ways (hash 232 ms, merge 432 ms, nested loop 641 ms), then flips to nested loop when a filter shrinks the outer to nine rows.

`SET enable_hashjoin = off` does not forbid the hash join: it adds a huge cost penalty (10^{10}) so that the planner falls back to merge join, and with `enable_mergejoin = off` too, to nested loop; when no alternative exists the penalised plan still runs, with absurd cost numbers that distort every estimate around it. The switches exist for one-query experiments, followed by `RESET ALL`.

Algorithm	Cost (pages)	Clearly best	Disaster
Block nested loop	$B_R + \lceil B_R/(M-2) \rceil B_S$	one input fits in a chunk; any condition	two large inputs, no index: 30,300 against 3,300
Index nested loop	$B_R + \lvert R \rvert (h + \text{matches})$	tiny outer, index on the inner key: `c.id = 42`	large outer: 20,000 descents, 641 ms against 232
Sort-merge	$\text{sort}(R) + \text{sort}(S) + B_R + B_S$	inputs already sorted, or sorted output wanted; ranges	neither sorted, little memory: two external sorts first
Hash	$B_R + B_S$ fits; $3(B_R + B_S)$ Grace	large equi-join, no useful order, build fits	non-equality; build $\gg$ memory (`Batches` $\gg 1$); skewed build key

Worked example: Mock v2 D2: `enable_nestloop = off` and `enable_mergejoin = off` for the whole database

Claim. Hash join is not always the best. **Mechanism.** Nested loop is best when the outer is a handful of rows and the inner has an index on the join key: `WHERE c.id = 42` costs one descent, about $1+(3+1)$ pages, where the hash join must read all 3,185 pages of `orders`; it is also the only algorithm for non-equi joins (`o.amount BETWEEN p.lo AND p.hi`). Merge join is best when both inputs arrive sorted on the key (index scans or an earlier sort): $B_R + B_S$, no hash table, streaming, sorted output, so an `ORDER BY cust_id` above it is free. **Consequences.** Every point query now reads whole tables through a hash join (a 1 ms lookup becomes a 60 ms scan), every sorted output is re-sorted, every join builds a hash table that competes for `work_mem` and spills under load. Reset both; if one report was helped, fix the bad estimate behind it (`ANALYZE`, `CREATE STATISTICS`).

Mock drill

Mock 1, 2.4: the table above as prose, one best case and one disaster per algorithm with a number each, then "why keep all three": condition, sizes, indexes, order, memory. **Mock v2 D2** is the example above in the rubric's shape: claim, mechanism (one situation per disabled algorithm), number ($1 + (h + 1)$ pages against 3,185), consequence (penalties, not prohibitions; the OLTP side regresses). **Mock v2 B3(d)**: "switching hash joins off tripled the report (32,080 against 9,840) for every join in the session; memory, not the algorithm, fixed it, and at $M = 100$ the two tie; re-enable it, keep the larger `work_mem`; a planner setting is not a fix for a correctness bug". Full chapter: §5.5 pages 10–11, classic P5 and P7.

6 Aggregation: Hash or Sort

`GROUP BY` faces the same question as a join: rows with the same key must be brought together, and the two answers are the same two. **HashAggregate** keeps one hash-table entry per group and updates the running aggregate as

rows stream in; it emits only at the end (a pipeline breaker) and holds one entry per group. **GroupAggregate** walks input sorted on the group key; a group is complete the moment the key changes, so it streams and holds one group at a time. The planner picks by the *number of groups*: few groups give a tiny hash table (`GROUP BY country`, 12 groups, 24 KB, 6 ms in the class example; 5 groups in mock v2 C1); groups near the row count would push the table past `work_mem`, so it sorts first (`GROUP BY name`, 19,104 groups: `Sort` then `GroupAggregate`, 26 ms). An index scan that delivers the key in order gives a GroupAggregate with no sort; `DISTINCT` is the same choice one level down.

Mock drill

Mock v2 C1: the HashAggregate node is blocking because no country total is final before the last joined row. Quiz A18, B8, C4: HashAggregate when groups are few, Sort + GroupAggregate when they are many or the input is already ordered; only the latter streams. Full chapter: §6 page 11.

Warning / Gotcha: The four mistakes that cost points

- $|R|$ **where** B_R **belongs.** Simple nested loop uses rows $(B_R + |R| \cdot B_S)$; block nested loop uses pages in chunks of $M - 2$. The 900,300 and 9,300 distractors of mock 1.5 are these slips.
- **Fan-in** M**, forgetting pass 0, counting reads only.** Fan-in is $M - 1$; passes are $1 + \lceil \log_{M-1} \text{runs} \rceil$; each pass costs $2B$. The run count is never the pass count.
- **"Doubling memory halves the sort."** Passes are a ceiling of a logarithm: doubling M removes at most one pass, and nothing below $M \geq B$ removes the last merge pass.
- **Hash join for everything.** Hashing finds equal keys only (no `<`); it costs $B_R + B_S$ only while the *build* side fits, else $3(B_R + B_S)$ with `Batches` > 1; a tiny indexed outer makes the nested loop hundreds of times cheaper.

Summary: The essential in five lines

1. Operators are iterators; tuples stream except through pipeline breakers (Sort, Hash build, HashAggregate), which materialise and may spill.
2. External sort: runs $= \lceil B/M \rceil$, fan-in $M - 1$, merge passes $= \lceil \log_{M-1} \text{runs} \rceil$, I/O $= 2B(1 + \text{merge passes})$. Two passes need $M \approx \sqrt{B}$; one pass needs $M \geq B$.
3. Block nested loop $B_R + \lceil B_R/(M-2) \rceil B_S$, small input outside; index nested loop $B_R + |R|(h + \text{matches})$, unbeatable for a tiny outer; both accept any condition.
4. Hash join builds on the smaller side: $B_R + B_S$ when it fits in `work_mem`, else Grace at $3(B_R + B_S)$ with `Batches` > 1. Sort-merge: two sorts plus $B_R + B_S$; free when the inputs arrive sorted; handles ranges.
5. No join dominates: condition, sizes, indexes, order and memory decide, so the optimiser keeps all three; `enable_*` are penalties. Aggregation mirrors joins: hash for few groups, sort for many.